Αναγνώριση Προτύπων - Εργαστήριο 1

Αναγνώριση φωνής με Κρυφά Μαρκοβιανά Μοντέλα και Αναδρομικά Νευρωνικά Δίκτυα

Κόρδας Νικόλαος (03121032)
Κριθαρίδης Κωνσταντίνος (03121045)

November 13, 2025

Βήμα 9

Πρώτα από όλα κάνουμε clone τα δεδομένα σε μορφή .wav από το repository της εργασίας στον υπολογιστή μας.

Κατόπιν, υλοποιούμε τη συνάρτηση *parse_free_digits* προκειμένου να διαβάσουμε σωστά τα αρχεία με τη βιβλιοθήκη librosa.

```
def parse_free_digits(directory):
    # Parse relevant dataset info
    files = glob(os.path.join(directory, "*.wav"))
    fnames = [f.split("/")[1].split(".")[0].split("_") for f in files]
    ids = [f[2] for f in fnames]
    y = [int(f[0]) for f in fnames]
    speakers = [f[1] for f in fnames]
    _, Fs = librosa.core.load(files[0], sr=None)

    def read_wav(f):
        wav, _ = librosa.core.load(f, sr=None)

        return wav

    # Read all wavs
    wavs = [read_wav(f) for f in files]

    # Print dataset info
    print("Total wavs: {}. Fs = {} Hz".format(len(wavs), Fs))

    return wavs, Fs, ids, y, speakers
```

Το αποτέλεσμα της κλήσης της συνάρτησης είναι:

Total wavs: 3000. Fs = 8000 Hz

Στην συνέχεια βασισμένοι στην υπολοποίηση του βοηθητικού κώδικα ορίζουμε τη δικιά μας συνάρτηση *split_free_digits* την οποία παρουσιάζουμε παρακάτω:

```
def extract_features(wavs, n_mfcc=6, Fs=8000):
    # Extract MFCCs for all wavs
    window = 30 * Fs // 1000
    step = window // 2
    frames = [
        librosa.feature.mfcc(
            y=wav, sr=Fs, n_fft=window, hop_length=window - step, n_mfcc=n_mfcc
        ).T
        for wav in tqdm(wavs, desc="Extracting mfcc features...")
```

```
    ]

    print("Feature extraction completed with {} mfccs per frame".format(n_mfcc)
    )

    return frames


def split_free_digits(frames, ids, speakers, labels):
    print("Splitting in train test split using the default dataset split")
    # Split to train-test
    X_train, y_train, spk_train = [], [], []
    X_test, y_test, spk_test = [], [], []
    test_indices = ["0", "1", "2", "3", "4"]

    for idx, frame, label, spk in zip(ids, frames, labels, speakers):
        if str(idx) in test_indices:
            X_test.append(frame)
            y_test.append(label)
            spk_test.append(spk)
        else:
            X_train.append(frame)
            y_train.append(label)
            spk_train.append(spk)

    return X_train, X_test, y_train, y_test, spk_train, spk_test
```

Τρέχουμε τον κώδικα για να επιβεβαιώσουμε την ορθή λειτουργία του:

```
└─ python3 lab.py
Total wavs: 3000. Fs = 8000 Hz
Splitting in train test split using the default dataset split
Train dataset's size is 2400
Train dataset has 60 samples of digit 0
Train dataset has 60 samples of digit 0
Train dataset has 60 samples of digit 1
Train dataset has 60 samples of digit 1
Train dataset has 60 samples of digit 2
Train dataset has 60 samples of digit 2
Train dataset has 60 samples of digit 3
Train dataset has 60 samples of digit 3
Train dataset has 60 samples of digit 4
Train dataset has 60 samples of digit 4
Train dataset has 60 samples of digit 5
Train dataset has 60 samples of digit 5
Train dataset has 60 samples of digit 6
Train dataset has 60 samples of digit 6
Train dataset has 60 samples of digit 7
Train dataset has 60 samples of digit 7
Train dataset has 60 samples of digit 8
Train dataset has 60 samples of digit 8
Train dataset has 60 samples of digit 9
Train dataset has 60 samples of digit 9
```

Βλέπουμε ότι το split που έχουμε κάνει έχει μέγεθος 2400, δηλαδή είναι πράγματι το 80% του 3000. Επιπλέον, είναι ισορροπημένο, αφού υπάρχει ίδιος αριθμός δειγμάτων ήχου σε κάθε κατηγορία - ψηφίο (60). Επομένως ως μετρική του *accuracy* θα είναι αρκετό για να αξιολογήσουμε την επίδοση των ταξινομητών μας (όπως είδαμε και στην προπαρασκευή, στην περίπτωση του ισορροπημένου dataset το F1-score ακολουθεί το ίδιο trend με το accuracy).

Αφού επιβεβαιώσαμε την ορθή λειτουργία της ρουτίνας μας αφαιρούμε - μετατρέπουμε σε σχόλιο τον κώδικα για το debugging.

Βήμα 10

Στο βήμα αυτό υλοποιούμε αναγνώριση ψηφίων από ομιλία με GMM-HMM. Η λειτουργία του μοντέλου φαίνεται διαγραμματικά παρακάτω:

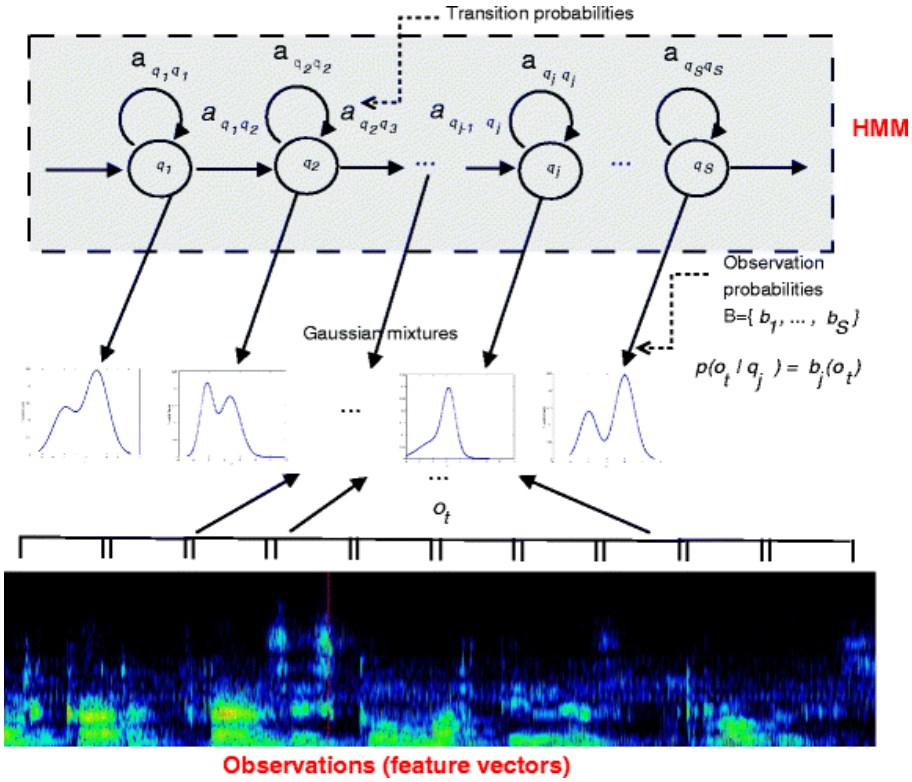


Figure 1: Image by [?]

Δημιουργούμε ένα HMM ανά ψηφίο, οπότε η έξοδος του i-οστού HMM θα είναι η πιθανότητα το σήμα να αντιστοιχεί στο i-οστό ψηφίο. Οι παρατηρήσεις σε κάθε state του HMM είναι ένα διαφορετικό mixture of gaussians. Αφού εκπαιδευτεί το GMM από μόνο του σε όλα τα δεδομένα, θα εκπαιδευτεί από κοινού το GMM-HMM μοντέλο και το μείγμα κατανομών ανά state θα γίνει fit στην παρατήρηση που συμβαίνει σε αυτό το state. Ιδανικά, θα θέλαμε εξ'αρχής να μην εκπαιδεύουμε το GMM σε όλα τα δεδομένα, αλλά μόνο σε αυτά που αντιστοιχούν στο χρονικό διάστημα που περιγράφεται από το state. Κάτι τέτοιο, όμως, είναι δυστυχώς δύσκολο να γίνει, αφού δεν είναι προκαθορισμένο για κάθε ψηφίο το πότε θα γίνει η μετάβαση από το ένα state στο επόμενο, οπότε θα αρκεστούμε στην διαφοροποίηση μεταξύ των GMMs που κάνει στη συνέχεια η από κοινού εκπαίδευση. Τέλος, σημειώνουμε ότι με το μείγμα γκαουσιανών μοντελοποιούμε την κατανομή των MFCCs ανά ψηφίο και state, αφού περιμένουμε πως η κάθε τέτοια κατανομή δεν θα είναι unimodal στον χώρο των MFCCs λόγω των διαφορών στην εκφώνηση του ίδιου ψηφίου μεταξύ διαφορετικών ομιλητών.

Η συνάρτηση train φαίνεται παρακάτω:

```
def train_hmms(train_dic, labels):
    hmms = {}  # create one hmm for each digit

    for dig in labels:
        X, _, _, _ = train_dic[dig]
        # TODO: YOUR CODE HERE
        emission_model = None
        if gmm: emission_model = initialize_and_fit_gmm_distributions(X,
    n_states=n_states, n_mixtures=n_mixtures)
        else: emission_model = initialize_and_fit_normal_distributions(X,
    n_states=n_states)
        hmms[dig] = train_single_hmm(X, emission_model, dig, n_states)
    return hmms
```

Την `initialize_and_fit_normal_distributions` καθώς είναι υποπερίπτωση του GMM για

n_mixtures = 1.

Η αρχική εκπαίδευση του GMM γίνεται ως εξής:

```
def initialize_and_fit_gmm_distributions(X, n_states, n_mixtures):
    # TODO: YOUR CODE HERE
    dists = []
    all_data = np.concatenate(X, dtype=np.float32) # Concatenate all frames from all samples into a large matrix, shape (N, D)
    D = all_data.shape[1]

    for _ in range(n_states):
        distributions = [Normal(means=np.random.randn(D).astype(np.float32), covs=np.ones(D, dtype=np.float32), covariance_type=covariance_type) for _ in range(n_mixtures)] # n_mixtures gaussian distributions
        a = GeneralMixtureModel(distributions, verbose=True, max_iter=n_iters, random_state=42, tol=tol).fit(all_data) # default log propability threshold is 0.1
        dists.append(a)
    return dists
```

Δηλαδή αρχικοποιούμε κάθε γκαουσιανή σε multivariate gaussian με μέσο ειλημμένο από D-διάστατη τυπική κανονική κατανομή, όπου D η διάσταση των δεδομένων, εδώ 13 για τα MFCCs, και με συνδιασπορά τον D-διάστατο ταυτοτικό πίνακα. Επίσης περιορίζουμε κατά την εκπαίδευση τον πίνακα συνδιασποράς να είναι διαγώνιος, ώστε να μειώσουμε τις παραμέτρους και τον χρόνο εκπαίδευσης αλλά και να έχουμε πιο απλές αναπαραστάσεις για τα δεδομένα μας.

Αντίστοιχα, το HMM το εκπαιδεύουμε ως εξής:

```
def train_single_hmm(X, emission_model, digit, n_states):
    A = initialize_transition_matrix(n_states)
    start_probs = initialize_starting_probabilities(n_states)
    end_probs = initialize_end_probabilities(n_states)
    data = [x.astype(np.float32) for x in X]

    model = DenseHMM(
        distributions=emission_model,
        edges=A,
        starts=start_probs,
        ends=end_probs,
        verbose=True,
        max_iter=n_iters,
        tol=tol,
        random_state=42
    ).fit(data)
    return model
```

Για πίνακα μετάβασης χρησιμοποιήσαμε τον $A = [\alpha_{i,j}]_{1 \le i,j \le n}$, όπου n = n_states το πλήθος καταστάσεων στα HMMs, με:

$$\alpha_{i,j} = \begin{cases} 1, & \text{if } j = i = n \\ 0.8 & \text{if } j = i \neq n \\ 0.2 & \text{if } j = i + 1 \\ 0 & \text{αλλιώς} \end{cases}$$

Οι τιμές 0.8 για την ανακύκλωση και 0.2 για την μετάβαση επιλέχθηκαν επειδή το μοντέλο πιο συχνά θα επιλέγει να μείνει στην ίδια κατάσταση, παρά να μεταβεί στην επόμενη. Επιπλέον, στον κώδικά μας, επειδή η βιβλιοθήκη pomegranate δημιουργούσε nan values από αριθμητική αστάθεια κατά την εκπαίδευση, βρήκαμε ότι αν στον τελευταίο κλάδο (otherwise) θέσουμε $\alpha_{i,j} = \varepsilon$ για κάποιο μικρό $\varepsilon > 0$ αντί για 0, σταματούσε το πρόβλημα αφού υπήρχε καλύτερη αριθμητική ευστάθεια.

Τέλος, αρχικοποιήσαμε τις πιθανότητες εκκίνησης ως 1 στην αρχική κατάσταση και 0 σε όλες τις άλλες, ενώ τις πιθανότητες τερματισμού ως 1 στην τελική κατάσταση και 0 σε όλες τις άλλες.

```
def initialize_starting_probabilities(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    pi = np.zeros(n_states, dtype=np.float32)
    pi[0] = np.float32(1.0)
    return pi


def initialize_end_probabilities(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    pi_end = np.zeros(n_states, dtype=np.float32)
    pi_end[-1] = np.float32(1.0)
    return pi_end
```

Βήμα 11

Βήμα 12

Βήμα 13

Βήμα 14